TÌM KIẾM SONG SONG

CƠ BẢN & NÂNG CAO

qua 8 bài Luogu 洛谷



Lê Thành Nam

Mỗi hàng là một truy vấn đang tự chặt nhị phân; vạch xanh là đoạn còn nghi ngờ, chấm hồng là mốc được hỏi. Mọi truy vấn hỏi trong cùng một lượt quét.

```
for (t = 1..T) { apply(t); for (q : giờ[t]) ok(q) ? hi[q] = t : lo[q] = t + 1; }
```

4 chương · 8 bài · 25 chương trình đã kiểm chứng và đo thời gian thật

1. Theo vòng
2. Fenwick · DSU
3. Cây phân đoạn · 2D
4. 整体二分 có cập nhật

LÊ THÀNH NAM

parallel binary search

MỤC LỤC

MỤC LỤC 1
GIỚI THIỆU 2
Chương 1. Từ một lần chặt nhị phân đến nhiều lần cùng lúc 4
Bài 1. Số nhỏ thứ k trong đoạn 5
Bài 2. Mưa sao băng 10
Chương 2. Quét theo thời gian: Fenwick và hợp nhất tập hợp 15
Bài 3. Bắn ván gỗ 16
Bài 4. Hành trình đóng dấu 21
Chương 3. Hàm kiểm tra phức tạp: cây phân đoạn và Fenwick hai chiều 26
Bài 5. Treo biển trên hàng rào 27
Bài 6. Số nhỏ thứ k trong hình chữ nhật 32
Chương 4. 整体二分 có cập nhật 36
Bài 7. Xếp hạng động 37
Bài 8. Truy vấn số lớn thứ k 42
PHỤ LỤC 47

Lê Thành Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Sách gồm 8 bài về **tìm kiếm song song** (parallel binary search, 整体二分) chọn từ kho đề Luogu (洛谷, luogu.com.cn), gồm cả các bài Codeforces và AtCoder được Luogu lưu trữ (mã bắt đầu bằng CF và AT). Đây là kĩ thuật dành cho tình huống *nhiều truy vấn, mỗi truy vấn tự chặt nhị phân được, nhưng mỗi lần kiểm tra lại quá đắt*: thay vì để từng truy vấn tự chặt, ta cho tất cả cùng chặt và dùng chung một lượt quét.

Sách phù hợp với học sinh đã vững chặt nhị phân, cây Fenwick, DSU và cây phân đoạn. Số bài ít, nhưng mỗi bài được giải theo nhiều cách (ngây thơ, tìm kiếm song song theo vòng, 整体二分 đệ quy và một cấu trúc dữ liệu trực tuyến để đối chiếu), đo thời gian thật và so sánh, để người đọc thấy rõ khi nào nên dùng cách nào.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc. **Hướng dẫn giải** đi qua nhiều tầng, từ ngây thơ đến tối ưu, kèm code chạy được, thời gian đo thật và hình minh hoạ.

> **Về cách viết đề**
>
> Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn, định dạng vào/ra và test ví dụ được giữ nguyên; mọi ví dụ đã được đối chiếu bằng cách chạy lời giải. Giới hạn thời gian ghi trong sách là giới hạn sách dùng để chấm các subtask, có thể khác giới hạn trên Luogu.

Cách giải

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng; hầu hết các bước đều có hình minh hoạ.

Phần	Nội dung
HIỂU ĐỀ	Đưa câu hỏi về dạng "thời điểm / giá trị nhỏ nhất làm điều kiện đúng" và chỉ ra điều kiện đơn điệu
NGÂY THƠ	Tầng 1: mô phỏng hoặc trả lời từng truy vấn độc lập, code và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một *câu hỏi then chốt*
CẢI TIẾN	Thường là tìm kiếm song song **theo vòng**: mỗi vòng quét lại toàn bộ thời gian một lần
LÓE SÁNG	Nhận xét quyết định: đổi vai, biến chiều cao thành thời gian, tách lệnh sửa thành xoá + thêm, ...
TỐI ƯU	Thường là 整体二分 **đệ quy**, hoặc một lời giải trực tuyến để đối chiếu; bảng so sánh mọi tầng trên mọi subtask
LƯU Ý	Những chỗ dễ sai: tràn số, mốc "không bao giờ", hoàn tác cấu trúc dữ liệu, thứ tự thời gian

Mỗi bài có ít nhất một hộp **Thiết kế lời giải** màu vàng, tóm tắt "thời gian" là gì, cấu trúc dữ liệu nào trả lời câu hỏi có/không, và câu hỏi đó cụ thể ra sao.

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tuỳ chọn -O2, chuẩn C++17) trên máy chủ Linux, tính cả thời gian đọc và ghi dữ liệu. Trên máy chấm Luogu các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

Subtask và các tầng lời giải

Mỗi bài có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, subtask cuối là ràng buộc gốc. Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được. Một vài kết quả đo khá bất ngờ (cây bền vững chậm hơn cách ngoại tuyến; cây Kruskal chậm hơn tìm kiếm song song), và sách giải thích lí do ngay tại chỗ.

Lê Thanh Nam

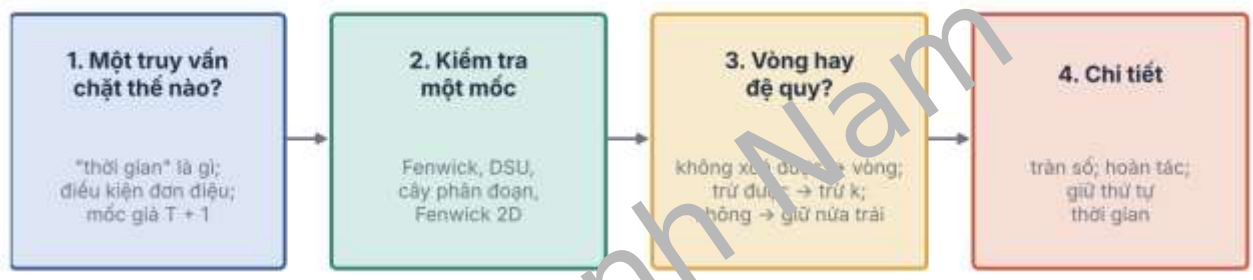
Sử dụng sách

Với mỗi bài, nên đọc đề, tự nghĩ ít nhất 20–30 phút rồi mới đọc hướng dẫn, và dừng lại ở mỗi "Câu hỏi then chốt" để tự trả lời trước khi đọc tiếp. Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 120 test ngẫu nhiên mỗi bài.

Nộp bài trên Luogu

- Đăng kí tài khoản tại luogu.com.cn (giao diện có thể chuyển sang tiếng Anh).
- Mở đề theo mã bài (ví dụ P3834, CF484E hoặc AT_agc002_d), chọn 提交答案 (nộp bài), chọn ngôn ngữ C++14 hoặc C++17 (bật O2 nếu muốn), dán code và nộp.
- Chương trình đọc từ bàn phím (stdin) và ghi ra màn hình (stdout).
- Với các bài mã CF và AT, Luogu gửi bài nộp sang Codeforces / AtCoder chấm hộ (remote judge); cần liên kết tài khoản theo hướng dẫn của Luogu.
- Kết quả: AC (đúng), WA (sai đáp án), TLE (quá thời gian), MLE (quá bộ nhớ), RE (lỗi khi chạy).

Bốn bước tư duy trước một bài tìm kiếm song song



Bốn bước xây dựng lời giải bằng tìm kiếm song song

Lê Thành Nam

- **Bước 1. Một truy vấn thì chặt nhị phân thế nào?** Xác định "thời gian" (thứ tự sự kiện, thứ tự giá trị) và điều kiện đơn điệu. Thêm mốc giả T + 1 nếu có thể không bao giờ thoả.
- **Bước 2. Kiểm tra một mốc bằng cấu trúc gì?** Fenwick (đếm), DSU (liên thông), cây phân đoạn (đoạn dài nhất), Fenwick 2D.
- **Bước 3. Theo vòng hay đệ quy?** Cấu trúc không xoá được (DSU) ⇒ theo vòng; điều kiện cộng trừ được ⇒ đệ quy và trừ k; không trừ được ⇒ đệ quy và giữ nửa trái.
- **Bước 4. Chi tiết.** Tràn số, hoàn tác đúng phần đã thêm, giữ thứ tự thời gian khi có cập nhật.

Cách	Chi phí	Dùng khi
Chặt nhị phân riêng từng truy vấn	$Q \cdot \log T \cdot$ (chi phí dựng lại)	ít truy vấn
Tìm kiếm song song theo vòng	$\log T \cdot (T + Q) \cdot$ (chi phí thao tác)	cấu trúc chỉ thêm (DSU), dễ viết
整体二分 đệ quy	$\log T \cdot (T + Q) \cdot$ (chi phí thao tác)	cần hoàn tác được; có cập nhật xen kẽ
Cấu trúc bền vững / lồng	$(T + Q) \cdot \log$ hoặc $\log^2$	bắt buộc trả lời trực tuyến

Bố cục sách

Chương	Chủ đề	Bài (mã Luogu)
1	Từ một lần chặt nhị phân đến nhiều lần cùng lúc	P3834, P3527
2	Quét theo thời gian: Fenwick và hợp nhất tập hợp	P7424, AT_agc002_d
3	Hàm kiểm tra phức tạp: cây phân đoạn và Fenwick hai chiều	CF484E, P1527
4	整体二分 có cập nhật	P2617, P3332

Chương 1. Từ một lần chặt nhị phân đến nhiều lần cùng lúc

Trọng tâm chương

Khi có rất nhiều truy vấn, mỗi truy vấn tự chặt nhị phân được, nhưng mỗi lần kiểm tra lại đắt. Chương xây dựng **tìm kiếm song song**: mọi truy vấn cùng hỏi trong một lượt quét, và phiên bản đệ quy của nó là 整体二分.

Kiến thức cần nhớ

- Đáp án của mỗi truy vấn là "thời điểm / giá trị đầu tiên mà điều kiện đúng", và điều kiện đơn điệu.
- Một vòng: xếp truy vấn theo mid, quét thời gian từ đầu, thêm dần sự kiện vào cấu trúc dữ liệu.
- Số vòng $\lceil \log_2(\text{số thời điểm}) \rceil$; mỗi vòng $O((\text{sự kiện} + \text{truy vấn}) \cdot \text{chi phí thao tác})$.
- Đệ quy (整体二分): chỉ thêm sự kiện nửa trái, chia truy vấn sang hai phía, trừ phần đã đếm.

Các bài trong chương

Bài	Tên bài	Mã Luogu	Nguồn	Số tầng
1	Số nhỏ thứ k trong đoạn 【模板】可持久化线段树 2	P3834	Luogu, bài mẫu	4
2	Mưa sao băng [POI2011] MET-Meteors	P3527	POI 2011, vòng 3	3

Lê Thành Nam

Bài 1. Số nhỏ thứ k trong đoạn

Luogu P3834 – 【模板】可持久化线段树 2

Nguồn: Luogu, bài mẫu · Giới hạn thời gian sách dùng: 1,0 giây · Bộ nhớ: 512 MB
Đề gốc:https://www.luogu.com.cn/problem/P3834

Đề bài

Cho dãy $a_1, a_2, \ldots, a_n$ và m truy vấn. Truy vấn (l, r, k) hỏi: trong các số $a_l, \ldots, a_r$, **số nhỏ thứ k** là bao nhiêu (đếm cả các số bằng nhau).

Dữ liệu vào

Dòng đầu chứa n, m. Dòng thứ hai chứa n số a_i. Mỗi dòng trong m dòng tiếp theo chứa l, r, k.

Kết quả

Với mỗi truy vấn in một dòng là đáp án.

Ràng buộc

- $1 \le n, m \le 2 \cdot 10^5$; $|a_i| \le 10^9$; $1 \le l \le r \le n$; $1 \le k \le r - l + 1$.

Ví dụ

Dữ liệu vào	Kết quả
5 5 25957 6405 15770 26287 26465 2 2 1 3 4 1 4 5 1 1 2 2 4 4 1	6405 15770 26287 25957 26287

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 1,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$n, m \le 5000$	30%
ST2	$n, m \le 2 \cdot 10^5$ (gốc)	70%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài: biến "số thứ k" thành câu hỏi có/không

Sắp xếp **toàn bộ dãy** tăng dần, gọi giá trị đứng ở vị trí t (đánh số từ 0) là "giá trị thứ t". Với một truy vấn (l, r, k), đặt

cnt(t) = số chỉ số $i \in [l, r]$ mà a_i nằm trong t + 1 giá trị nhỏ nhất của cả dãy.

Khi t tăng, cnt(t) chỉ tăng hoặc giữ nguyên. Đáp án là giá trị thứ t với **t nhỏ nhất mà cnt(t) ≥ k**. Đây đúng là dạng "tìm vị trí đầu tiên mà điều kiện đúng" của chặt nhị phân.

Lê Thành Nam

t	0	1	2	3	4
giá trị thứ t	6405	15770	25957	26287	26465
cnt(t) trong [1, 3]	1	2	3	3	3
cnt(t) ≥ 2 ?	sai	đúng	đúng	đúng	đúng

↑ t nhỏ nhất cho "đúng": đáp án 15770

Hình 1.1. *Truy vấn (l, r, k) = (1, 3, 2) trên ví dụ: dãy đúng/sai đơn điệu, đáp án là giá trị đầu tiên cho "đúng".*

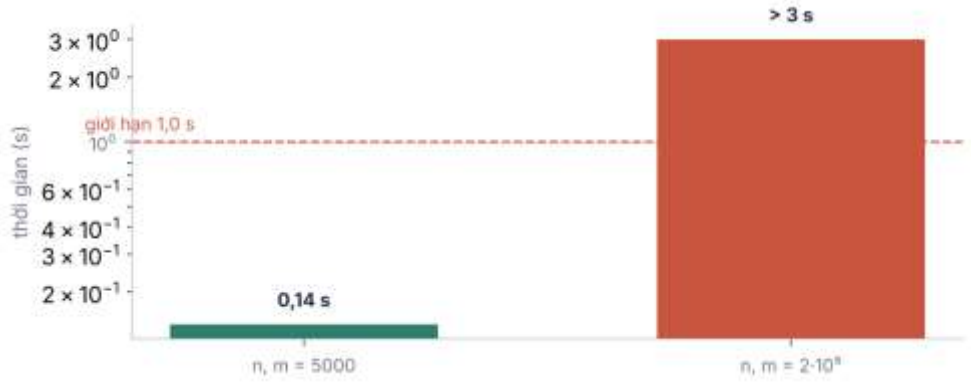
NGÂY THƠ Ý tưởng ngây thơ: chép đoạn rồi chọn

Mỗi truy vấn chép đoạn [l, r] ra và dùng nth_element (chọn phần tử thứ k trong thời gian trung bình tuyến tính).

Code. Tầng 1 – Chép đoạn + nth_element

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> a(n), b;
    for (auto &x : a) scanf("%d", &x);
    while (m--) {
        int l, r, k;
        scanf("%d %d %d", &l, &r, &k);
        b.assign(a.begin() + l - 1, a.begin() + r);          // chép đoạn [l, r]
        nth_element(b.begin(), b.begin() + k - 1, b.end());  // O(độ dài) trung bình
        printf("%d\n", b[k - 1]);
    }
}
```

BẾ TẮC Bế tắc: m đoạn dài, mỗi đoạn tốn O(n)



Hình 1.2. *Tầng 1 trên các nhóm test (thời gian đo thật, trục log)*

Đoạn dài tới $2 \cdot 10^5$, nhân với $2 \cdot 10^5$ truy vấn là $4 \cdot 10^{10}$ phép toán.

> **Câu hỏi then chốt**
>
> Chặt nhị phân **một** truy vấn thì mỗi bước phải đếm cnt(mid): mất O(n), lại còn chậm hơn nth_element. Nhưng nếu **mọi truy vấn cùng hỏi một lượt**, có thể đếm chung cho tất cả không?

Lê Thành Nam

CẢI TIẾN 1 Cải tiến 1: tìm kiếm song song theo vòng

Mỗi truy vấn q giữ đoạn nghi ngờ $[lo_q, hi_q]$ (ban đầu $[0, n-1]$). Mỗi **vòng**:

1. mỗi truy vấn chưa xong chọn $mid_q = \lfloor (lo_q + hi_q)/2 \rfloor$ và được bỏ vào "giỏ" mid_q;
2. quét t = 0, 1, …, n − 1: **thêm** vị trí của giá trị thứ t vào cây Fenwick (đánh dấu 1 tại vị trí đó), rồi trả lời mọi truy vấn trong giỏ t: cnt = tổng Fenwick trên [l, r];
3. cnt ≥ k thì $hi_q = mid_q$, ngược lại $lo_q = mid_q + 1$.

Mấu chốt: lúc trả lời giỏ t, Fenwick chứa **đúng** t + 1 giá trị nhỏ nhất. Một lần quét O(n log n) phục vụ cùng lúc tất cả các truy vấn. Mỗi vòng thu hẹp mọi đoạn một nửa, nên sau $\lceil \log_2 n \rceil$ vòng là xong.



Hình 1.3. *Các vòng trên ví dụ: sau 3 vòng mọi truy vấn đều chốt đáp án.*

Thiết kế lời giải – Tìm kiếm song song (theo vòng)	
Trạng thái	$[lo_q, hi_q]$ cho mỗi truy vấn
Một vòng	xếp truy vấn theo mid, quét "thời gian" t từ nhỏ đến lớn, thêm dần vào cấu trúc dữ liệu
Câu hỏi	cnt(mid) ≥ k? Trả lời bằng Fenwick
Số vòng	$\lceil \log_2 n \rceil$, mỗi vòng O((n + m) log n)

Code. Tầng 2 – Tìm kiếm song song theo vòng

```
#include <bits/stdc++.h>
using namespace std;
int n, m;
vector<int> bit;
void add(int i) { for (; i <= n; i += i & -i) bit[i]++; }
int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += bit[i]; return s; }
int main() {
    scanf("%d %d", &n, &m);
    vector<int> a(n + 1), ord(n);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    iota(ord.begin(), ord.end(), 1);
    sort(ord.begin(), ord.end(), [&](int x, int y) { return a[x] < a[y]; });  // ord[t]: vị trí của số nhỏ thứ t
    vector<int> L(m), R(m), K(m), lo(m, 0), hi(m, n - 1);
    for (int q = 0; q < m; q++) scanf("%d %d %d", &L[q], &R[q], &K[q]);
    vector<int> head(n), nxt(m);
    while (true) {
        // mỗi vòng: mọi truy vấn chưa xong cùng hỏi "đáp án ≤ mid?"
        fill(head.begin(), head.end(), -1);
        bool any = false;
        for (int q = 0; q < m; q++)
            if (lo[q] < hi[q]) {
                int mid = (lo[q] + hi[q]) / 2;
                nxt[q] = head[mid]; head[mid] = q; any = true;   // xếp truy vấn vào "giỏ" mid
            }
        if (!any) break;
```

Lê Thành Nam

```
        bit.assign(n + 1, 0);
        for (int t = 0; t < n; t++) {
            add(ord[t]);                                    // thêm số nhỏ thứ t
            for (int q = head[t]; q != -1; q = nxt[q]) {
                int c = sum(R[q]) - sum(L[q] - 1);          // số phần tử ≤ giá trị thứ t trong [L, R]
                if (c >= K[q]) hi[q] = t; else lo[q] = t + 1;
            }
        }
    }
    for (int q = 0; q < m; q++) printf("%d\n", a[ord[lo[q]]]);
}
```

LÓE SÁNG Lóe sáng: chia để trị trên giá trị

Cùng ý đó có thể viết **đệ quy**: hàm solve(vl, vr, Q) biết đáp án của mọi truy vấn trong tập Q nằm trong đoạn chỉ số giá trị [vl, vr]. Lấy mid, chỉ thêm các giá trị thứ vl..mid vào Fenwick, đếm c cho từng truy vấn:

- k ≤ c: đáp án ở nửa trái;
- ngược lại đáp án ở nửa phải, và **trừ k đi c** (c số ấy đã bị "vượt qua").

Xong thì xoá các giá trị vừa thêm và đệ quy hai nửa. Mỗi giá trị được thêm ở đúng một nút trên mỗi tầng đệ quy, mỗi truy vấn cũng vậy, nên tổng là O((n + m) log n) thao tác Fenwick. Kĩ thuật này người Trung Quốc gọi là 整体二分 ("chặt nhị phân tổng thể"). Nó tránh việc mỗi vòng phải quét lại cả n giá trị như cách 1.

Code. Tầng 3 – Chia để trị trên giá trị (整体二分)

```
#include <bits/stdc++.h>
using namespace std;
int n, m;
vector<int> bit, a, ord, L, R, K, ans;
void add(int i, int v) { for (; i <= n; i += i & -i) bit[i] += v; }
int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += bit[i]; return s; }
// đáp án của mọi truy vấn trong qs[ql..qr) nằm trong đoạn chỉ số giá trị [vl, vr]
void solve(int vl, int vr, vector<int> &qs, int ql, int qr) {
    if (ql >= qr) return;
    if (vl == vr) { for (int i = ql; i < qr; i++) ans[qs[i]] = a[ord[vl]]; return; }
    int mid = (vl + vr) / 2;
    for (int t = vl; t <= mid; t++) add(ord[t], 1);          // chỉ thêm nửa trái của đoạn giá trị
    vector<int> le, ri;
    for (int i = ql; i < qr; i++) {
        int q = qs[i], c = sum(R[q]) - sum(L[q] - 1);
        if (K[q] <= c) le.push_back(q);
        else { K[q] -= c; ri.push_back(q); }                  // trừ đi phần đã đếm ở nửa trái
    }
    for (int t = vl; t <= mid; t++) add(ord[t], -1);          // xoá để BIT sạch cho lời gọi khác
    int p = ql;
    for (int q : le) qs[p++] = q;
    for (int q : ri) qs[p++] = q;
    int split = ql + le.size();
    solve(vl, mid, qs, ql, split);
    solve(mid + 1, vr, qs, split, qr);
}
int main() {
    scanf("%d %d", &n, &m);
    a.resize(n + 1); ord.resize(n); bit.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    iota(ord.begin(), ord.end(), 1);
    sort(ord.begin(), ord.end(), [&](int x, int y) { return a[x] < a[y]; });
    L.resize(m); R.resize(m); K.resize(m); ans.resize(m);
    vector<int> qs(m);
    for (int q = 0; q < m; q++) { scanf("%d %d %d", &L[q], &R[q], &K[q]); qs[q] = q; }
    solve(0, n - 1, qs, 0, m);
    for (int q = 0; q < m; q++) printf("%d\n", ans[q]);
}
```

TỐI ƯU Đối chiếu với cây phân đoạn bền vững

Tên bài là "cây phân đoạn bền vững" (主席树): lưu n phiên bản cây đếm theo giá trị, truy vấn đi xuống bằng hiệu hai phiên bản root[r] − root[l − 1]. Đây là lời giải trực tuyến O(log n) mỗi truy vấn nhưng tốn khoảng 20·n nút bộ nhớ. Hai cách ngoại tuyến ở trên chỉ cần O(n + m) bộ nhớ và là bước đệm cho các bài sau, nơi không có cấu trúc bền vững nào tiện như vậy.

Lê Thành Nam

Kết quả đo thật khá bất ngờ: trên test lớn, cây bền vững chạy **chậm hơn** hai cách ngoại tuyến (xem bảng). Lí do là mỗi truy vấn nhảy qua khoảng 2·18 nút nằm rải rác trong $4 \cdot 10^6$ nút, gần như lần nào cũng trượt bộ nhớ đệm. Trong khi đó, Fenwick của cách ngoại tuyến chỉ có $2 \cdot 10^5$ phần tử và nằm gọn trong bộ nhớ đệm. Độ phức tạp nhỏ hơn chưa chắc đã chạy nhanh hơn.

Code. Tầng 4 – Cây phân đoạn bền vững (đối chiếu)

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MX = 200005 * 20;
int lc[MX], rc[MX], cnt[MX], tot = 0;
// phiên bản mới = phiên bản cũ + một số ở vị trí giá trị p (chỉ tạo log n nút mới)
int ins(int old, int l, int r, int p) {
    int u = ++tot;
    lc[u] = lc[old]; rc[u] = rc[old]; cnt[u] = cnt[old] + 1;
    if (l < r) {
        int mid = (l + r) / 2;
        if (p <= mid) lc[u] = ins(lc[old], l, mid, p);
        else rc[u] = ins(rc[old], mid + 1, r, p);
    }
    return u;
}
// số nhỏ thứ k trong (phiên bản v) − (phiên bản u): đi xuống một nhánh mỗi tầng
int kth(int u, int v, int l, int r, int k) {
    if (l == r) return l;
    int mid = (l + r) / 2, c = cnt[lc[v]] - cnt[lc[u]];
    if (k <= c) return kth(lc[u], lc[v], l, mid, k);
    return kth(rc[u], rc[v], mid + 1, r, k - c);
}
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> a(n + 1), vals;
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]), vals.push_back(a[i]);
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    int V = vals.size();
    vector<int> root(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int p = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
        root[i] = ins(root[i - 1], 0, V - 1, p);
    }
    while (m--) {
        int l, r, k;
        scanf("%d %d %d", &l, &r, &k);
        printf("%d\n", vals[kth(root[l - 1], root[r], 0, V - 1, k)]);
    }
}
```

Tầng	Ý tưởng	Độ phức tạp	n, m = 5000 (ST1)	n, m = $2 \cdot 10^5$ (ST2)	Điểm
1	Chép đoạn + nth_element	$O(n \cdot m)$	0,14 s	quá giờ	**30**
2	Tìm kiếm song song theo vòng	$O((n + m) \log^2 n)$	0,01 s	0,42 s	**100**
3	Chia để trị trên giá trị (整体二分)	$O((n + m) \log^2 n)$	0,01 s	0,37 s	**100**
4	Cây phân đoạn bền vững (đối chiếu)	$O((n + m) \log n)$	9,1 ms	0,82 s	**100**

Thời gian đo thật trên test lớn nhất của từng subtask; giới hạn 1,0 giây. Ô đỏ: quá giờ, sai hoặc quá bộ nhớ. Cột "Điểm" cộng phần trăm của các subtask mà tầng đó vượt qua.

LƯU Ý Những chỗ dễ sai

- Chặt nhị phân trên **chỉ số trong mảng đã sắp xếp** (0..n − 1) chứ không trên giá trị 10^9: số vòng giảm từ 31 xuống 18 và không cần rời rạc hoá riêng. Các giá trị bằng nhau vẫn đúng vì cnt đếm theo chỉ số.
- Ở cách đệ quy, quên "k −= c" khi rẽ phải là lỗi kinh điển.
- Mỗi lần đệ quy phải **xoá sạch** những gì đã thêm vào Fenwick. Nếu khởi tạo lại cả cây ở mỗi nút đệ quy thì độ phức tạp thành $O(n^2)$.
- Tìm kiếm song song chỉ dùng được khi trả lời **ngoại tuyến** (biết trước mọi truy vấn).

Le Thành Nam